

ALL ABOUT DATA
ENGINEERING

CLOUD DATA ENGINEERING ARCHITECTURES



Different ETL (Extract, Transform, Load) Architecture Designs

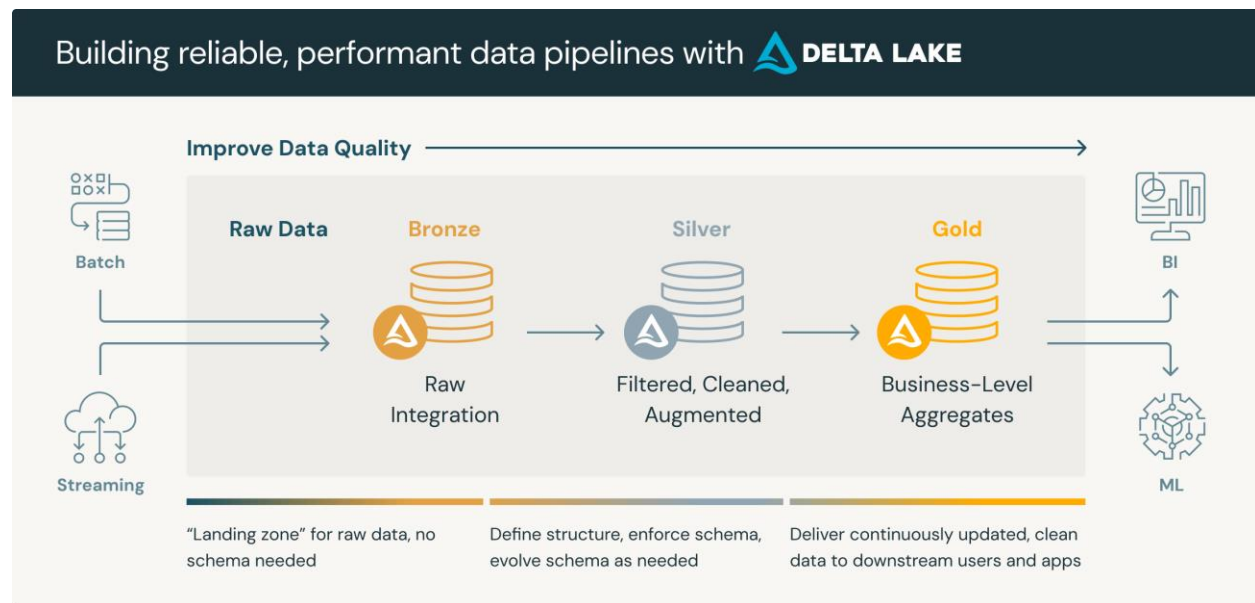
Introduction

Selecting the appropriate architecture for your data requirements is a critical initial step in establishing your project. Various ETL architectures are available to address diverse business needs and data complexities. This article explores common ETL architecture patterns and their best-fit scenarios, including the Medallion Architecture, Lambda Architecture, Kappa Architecture, Data Vault Architecture, Kimball's Dimensional Data Warehouse, Inmon's Corporate Information Factory, Lakehouse Architecture, and various ETL Pattern Variations. Each architecture is designed to tackle specific data management challenges, offering distinct advantages and use cases.

Medallion Architecture

The Medallion Architecture represents a layered approach to data processing that is frequently employed in data lakes. This architecture is designed to address the complexities of managing large volumes of diverse data by organizing it into progressively refined layers. The primary objective is to improve data quality and traceability while providing a clear framework for data governance.

Architecture and Layers



Ref: <https://www.databricks.com/glossary/medallion-architecture>

Bronze Layer

The Bronze Layer serves as the initial stage of data ingestion. In this layer, raw data is collected from multiple sources, including point-of-sale systems, online transactions, IoT devices, and other data-generating systems. This raw data is stored in its original format within a data lake, which allows for the storage of large volumes of diverse data types without immediate transformation.

Silver Layer

The Silver Layer focuses on cleaning and enriching the raw data ingested in the Bronze Layer. During this phase, data undergoes a series of transformations designed to improve its quality and usability. Key processes include the removal of duplicates, standardization of data formats, correction of errors, and integration of additional contextual information, such as customer demographics or product metadata.

Gold Layer

The Gold Layer represents the final stage in the Medallion Architecture, where data is aggregated and refined for specific business applications. This layer focuses on creating business-specific datasets that are optimized for reporting, dashboarding, and advanced analytics. The data in the Gold Layer is highly curated and often tailored to meet the needs of various business units, such as finance, marketing, and operations.

Possible Scenario

Scenario: A retail company wants to manage and analyze large volumes of transaction data from multiple stores and online sales channels.

Bronze Layer: Raw transaction data from point-of-sale systems and online transactions are ingested into a data lake.

Silver Layer: The raw data is cleaned, removing duplicates and standardizing formats. Enriched with additional data such as customer information and product details.

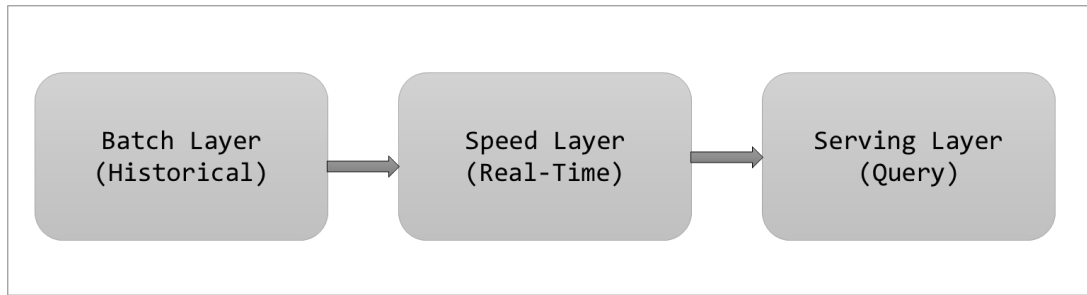
Gold Layer: Aggregated data is prepared for business-specific analysis, such as sales reports, inventory management, and customer behavior analysis.

Lambda Architecture

The Lambda Architecture is a data processing paradigm designed to handle massive quantities of data by leveraging both batch and real-time processing methods. It addresses the need for real-time analytics while simultaneously providing the capability to perform in-depth historical data analysis. This architecture is particularly effective in scenarios where both low-latency updates and comprehensive data processing are required.

Lambda Architecture divides data processing into three main components: the Batch Layer, the Speed Layer, and the Serving Layer. Each component plays a specific role in ensuring that data is processed efficiently and made available for querying in a timely manner.

Architecture and Layers



Ref: <https://www.databricks.com/glossary/lambda-architecture>

Batch Layer

The Batch Layer is responsible for processing and storing large volumes of historical data. This component performs comprehensive data processing tasks that can tolerate higher latencies, such as computing long-term aggregates, trends, and patterns. The processed data is stored in a batch view, which provides a consolidated view of the historical data.

Speed Layer

The Speed Layer handles real-time data streams, ensuring that recent data is processed with minimal latency. This component complements the Batch Layer by providing immediate insights and updates based on the latest data. The Speed Layer is typically built using stream processing frameworks such as Apache Storm, Flink, or Kafka Streams.

The primary goal of the Speed Layer is to process real-time data quickly and produce incremental updates that can be combined with the batch view in the Serving Layer. This ensures that users have access to both the most recent data and the historical data processed by the Batch Layer.

Serving Layer

The Serving Layer merges the batch view and the real-time view to provide a comprehensive and up-to-date view of the data. This layer supports queries that require access to both historical and real-time data, enabling users to perform complex analytical tasks.

The Serving Layer is designed to optimize query performance by integrating the outputs from the Batch Layer and the Speed Layer. This component ensures that users can retrieve accurate and timely insights from the data, regardless of its origin.

Possible Scenario

Scenario: A financial services company needs to process and analyze stock market data in real-time while also performing batch processing for historical trend analysis. Aim is to provide the traders with real-time market insights while also enabling analysts to study long-term market trends.

Batch Layer: Historical stock data is processed in large batches to compute long-term trends and patterns.

Speed Layer: Real-time stock price data is processed for immediate alerts and decisions.

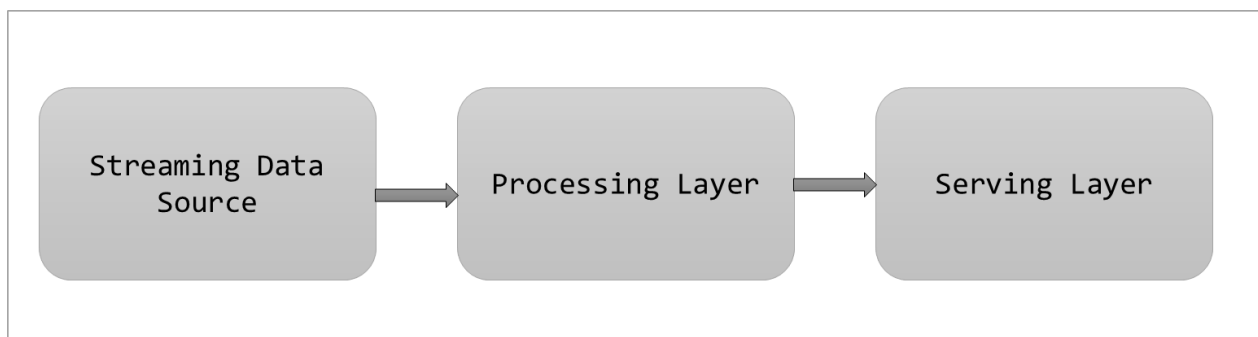
Serving Layer: Combines real-time data with batch data to provide a comprehensive view for analytics and reporting.

Kappa Architecture

The Kappa Architecture is a streamlined version of the Lambda Architecture, designed to handle real-time data processing exclusively. It eliminates the batch processing component, focusing solely on processing data streams in real-time. This architecture is particularly suited for use cases where historical data processing is not required, and the primary focus is on low-latency data ingestion and processing.

Kappa Architecture simplifies the data pipeline by using a single streaming layer to ingest, process, and store data. This approach reduces the complexity of managing separate batch and speed layers, making it easier to maintain and scale.

Architecture and Layers



Streaming Layer

The Streaming Layer is the core component of the Kappa Architecture, responsible for ingesting and processing data in real-time. This layer leverages stream processing frameworks like Apache Kafka, Apache Flink, or Apache Pulsar to handle continuous data streams.

The primary function of the Streaming Layer is to process incoming data events as they occur, applying transformations and computations on-the-fly. This ensures that data is processed with minimal latency and made available for immediate use.

Serving Layer

The Serving Layer in the Kappa Architecture provides a real-time view of the processed data. This layer stores the output of the Streaming Layer and supports real-time queries and analytics. The Serving Layer is designed to handle high query loads and deliver low-latency responses.

The integration between the Streaming Layer and the Serving Layer ensures that users always have access to the most current data, enabling real-time decision-making and analytics.

Possible Scenario

Scenario: A social media platform needs to monitor and analyze user activity data in real-time to detect trending topics and respond to user interactions immediately.

Streaming Layer: Ingests real-time user activity data (likes, shares, comments) and processes it to identify trends and generate metrics.

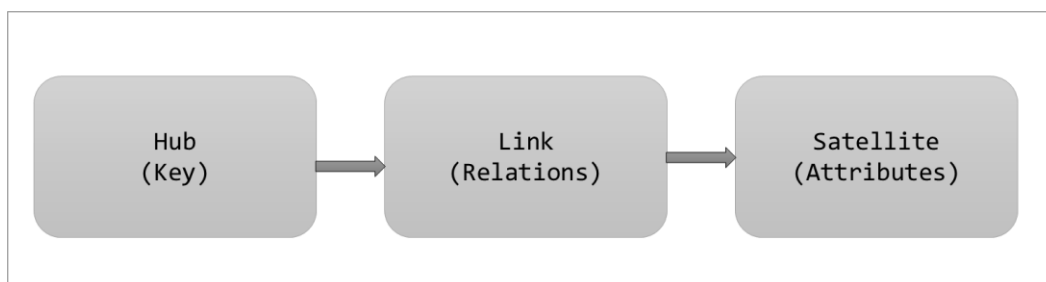
Serving Layer: Provides real-time analytics dashboards and notifications for trending topics and user engagement.

Data Vault Architecture

Data Vault Architecture is a data modeling technique designed to store historical data in a way that supports auditing and traceability. This architecture is built to handle large-scale data warehousing requirements, providing a flexible and scalable framework for managing historical data.

Data Vault Architecture consists of three primary components: Hubs, Links, and Satellites. These components work together to capture the historical state of data, maintain relationships between data entities, and store descriptive attributes.

Architecture and Layers



Hub

The Hub component contains unique business keys that represent core business entities. Hubs are designed to provide a consistent and immutable representation of business entities, ensuring data integrity and uniqueness. Each Hub table stores the primary keys for the business entities and their associated metadata, such as load timestamps and source identifiers.

Link

The Link component represents relationships between Hubs. Links capture the associations between business entities, maintaining the historical context of these relationships. Each Link table stores the foreign keys from the related Hubs, along with metadata that tracks the creation and modification of these relationships.

Satellite

The Satellite component stores descriptive attributes and historical data for the business entities represented in the Hubs. Satellites capture the changes to the attributes over time, allowing for detailed auditing and historical analysis. Each Satellite table is linked to a Hub or Link table and includes metadata that tracks the timing and source of the data changes.

Possible Scenario

Scenario: A healthcare provider needs to store and manage patient records, treatment histories, and medical billing information with strict auditing and historical tracking requirements.

Hub: Stores unique patient IDs and primary keys for treatments and billing.

Link: Represents relationships between patients, treatments, and billing records.

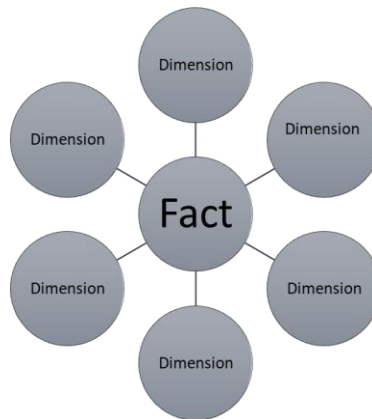
Satellite: Stores descriptive attributes and historical data such as treatment details, billing amounts, and changes over time.

Kimball's Dimensional Data Warehouse

Kimball's Dimensional Data Warehouse is based on Ralph Kimball's dimensional modeling approach, which focuses on creating user-friendly data structures for querying and reporting. This architecture emphasizes simplicity and ease of use, making it accessible to business users and analysts who need to perform ad-hoc queries and generate reports.

The Dimensional Data Warehouse architecture organizes data into Fact and Dimension tables, which are designed to support efficient querying and analysis. Fact tables store quantitative data, while Dimension tables store descriptive attributes related to the facts.

Architecture and Layers



Fact Tables

Fact tables are the core components of the Dimensional Data Warehouse, storing quantitative data for analysis. These tables contain measures such as sales revenue, transaction counts, and other numerical metrics. Fact tables are designed to support aggregation and summarization, enabling users to analyze data across various dimensions.

Dimension Tables

Dimension tables store descriptive attributes that provide context to the measures in the Fact tables. These attributes include information such as product names, customer demographics, dates, and locations. Dimension tables are structured to support hierarchical and categorical data, making it easy to filter, group, and drill down into the data during analysis.

Possible Scenario

Scenario: A marketing agency needs to analyze campaign performance, customer interactions, and sales data to optimize marketing strategies.

Fact Tables: Store quantitative data such as campaign impressions, clicks, and sales revenue.

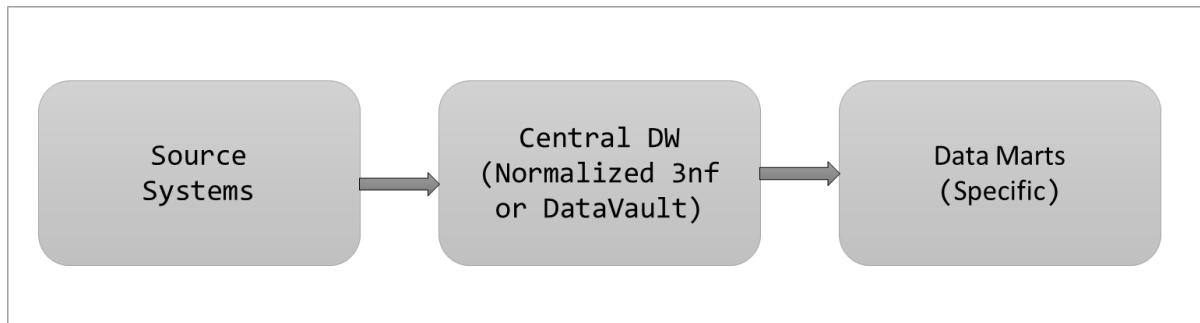
Dimension Tables: Store descriptive attributes like campaign names, dates, customer demographics, and product categories.

Inmon's Corporate Information Factory

Inmon's Corporate Information Factory (CIF) is based on Bill Inmon's approach to data warehousing, which emphasizes a centralized data warehouse with normalized data structures. This architecture is designed to provide a robust, scalable, and flexible data warehouse that integrates data from various sources and supports enterprise-wide reporting and analysis.

The Corporate Information Factory consists of a Centralized Data Warehouse and Data Marts. The Centralized Data Warehouse stores normalized data, ensuring data consistency and integration. Data Marts are denormalized subsets of data tailored to specific business areas or departments.

Architecture and Layers



Centralized Data Warehouse

The Centralized Data Warehouse is the core component of the Corporate Information Factory, storing normalized data from various source systems. This component ensures data consistency and integration across the organization, providing a single source of truth for enterprise data.

Data Marts

Data Marts are denormalized subsets of data extracted from the Centralized Data Warehouse. These components are designed to support specific business areas or departments, such as sales, finance, or human resources. Data Marts provide simplified and optimized data structures for efficient querying and analysis.

Possible Scenario

Scenario: A multinational corporation needs a robust data warehouse to integrate data from various departments (finance, HR, sales) for enterprise-wide reporting and analysis.

Centralized Data Warehouse: Stores normalized data from all departments, ensuring consistency and integration.

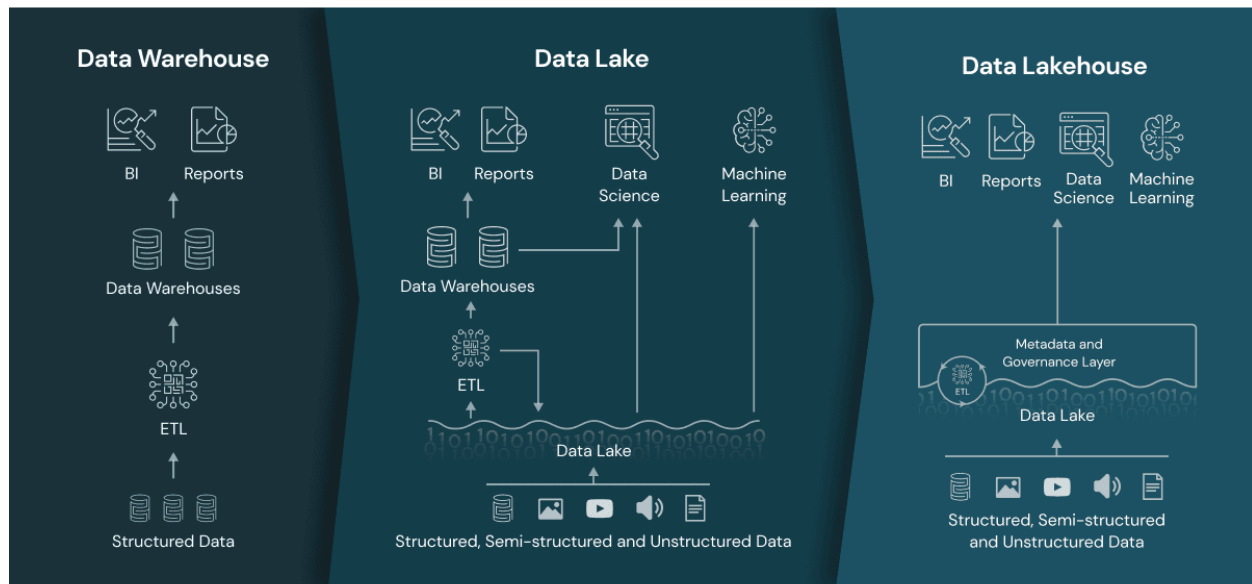
Data Marts: Specific data subsets for individual departments, such as sales data mart, finance data mart, and HR data mart.

Lakehouse Architecture

The Lakehouse Architecture is a modern data architecture that combines the flexibility of data lakes with the data management capabilities of data warehouses. This hybrid approach aims to provide a

unified platform for managing structured, semi-structured, and unstructured data, supporting both analytics and transactional workloads.

Architecture and Layers



Ref: <https://www.databricks.com/blog/2020/01/30/what-is-a-data-lakehouse.html>

Data Lake

The Data Lake component stores raw and semi-structured data, including log files, JSON data, and CSV files. This component leverages distributed storage systems like Amazon S3, Azure Data Lake, or Hadoop Distributed File System (HDFS) to store large volumes of diverse data types. The Data Lake provides a scalable and cost-effective storage solution for ingesting and managing raw data.

Data Warehouse

The Data Warehouse component manages structured data and supports ACID transactions for business-critical applications. This component leverages relational database management systems (RDBMS) or data warehouse solutions like Amazon Redshift, Google BigQuery, or Snowflake to enforce schema, ensure data consistency, and provide robust query capabilities. The Data Warehouse integrates with the Data Lake, allowing for seamless data movement and processing between the two components.

Possible Scenario

Scenario: A tech company wants to manage both structured data (e.g., sales records) and unstructured data (e.g., log files) efficiently, leveraging the benefits of both data lakes and data warehouses.

Data Lake: Stores raw and semi-structured data such as log files, JSON data, and CSV files.

Data Warehouse: Manages structured data and supports ACID transactions for business-critical applications.

ETL Pattern Variations

Batch ETL

Batch ETL is a traditional data processing approach where data is extracted, transformed, and loaded in scheduled batches. This method processes data at regular intervals, typically during off-peak hours, to reduce the impact on operational systems. Batch ETL is suitable for scenarios where real-time data processing is not critical, and data updates can be performed periodically.

Possible Scenario

Scenario: A university processes student enrollment data and course registrations nightly to update the student information system.

Implementation: Nightly jobs extract data from registration systems, transform it by cleaning and validating, and load it into the central student information system.

Real-time ETL

Real-time ETL processes data and loads it in real-time as it is ingested. This approach ensures that data is processed and made available immediately, supporting applications that require instant data updates. Real-time ETL leverages streaming data platforms and real-time processing frameworks to handle continuous data ingestion and transformation.

Possible Scenario

Scenario: An e-commerce website updates product inventory in real-time to reflect purchases and stock levels accurately.

Implementation: Streams data from purchase transactions and inventory updates, transforming it in real-time to update the product inventory database.

Micro-batch ETL

Micro-batch ETL processes small batches of data at frequent intervals, blending batch and real-time approaches. This method provides a balance between the efficiency of batch processing and the timeliness of real-time processing. Micro-batch ETL is designed to handle near-real-time data updates without the complexity and resource demands of continuous streaming.

Possible Scenario

Scenario: A news aggregator processes incoming news articles every few minutes to update its website with the latest content. Balancing near-real-time updates with efficient resource use for handling large volumes of incoming news articles.

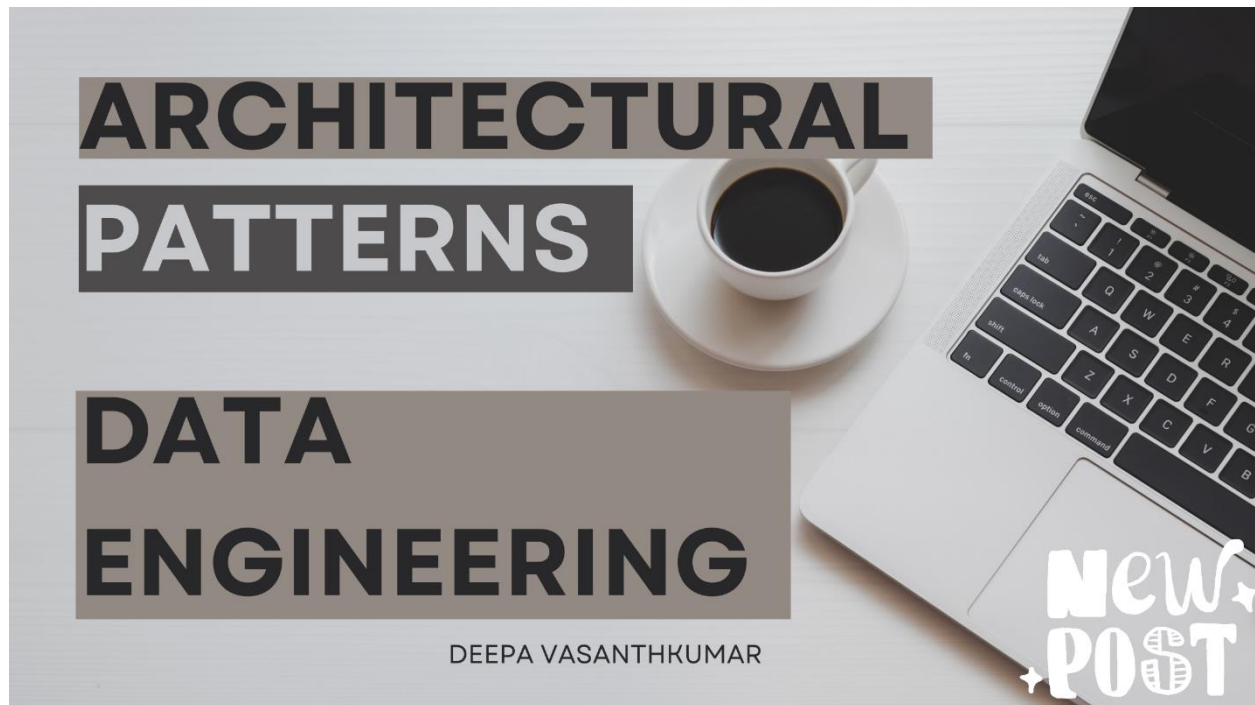
Implementation: Ingests new articles in micro-batches, transforms them (e.g., tagging, categorizing), and loads them into the content management system every few minutes.

Summary

This **article** presents a comprehensive overview of ETL architecture designs, each tailored to specific data management needs and organizational goals. The Medallion Architecture uses layered data processing for data lakes, while the Lambda Architecture combines batch and real-time processing for financial market data. The Kappa Architecture focuses on real-time processing for social media activity monitoring, and the Data Vault Architecture supports historical data storage with auditing for healthcare records. Kimball's Dimensional Data Warehouse provides user-friendly analytics for marketing agencies, and Inmon's Corporate Information Factory offers a centralized data warehouse for multinational corporations. The Lakehouse Architecture unifies data management for structured and unstructured data in tech companies. ETL pattern variations, including batch, real-time, and micro-batch ETL, cater to different data processing needs, from nightly updates to real-time inventory management.

Each architecture provides unique methods for handling ETL processes, ensuring data quality, reliability, and performance, thus supporting various business needs and data complexities effectively.

Exploring Architectural Patterns in Data Engineering Projects



Data engineering is a critical component of any data-driven organization, enabling the collection, transformation, and management of data to support analytics and decision-making. As data engineering projects vary widely in scope and complexity, different architectural patterns are employed to meet specific requirements and challenges. In this blog post, we will explore some of the most common architectural patterns in data engineering, discussing their key features, use cases, and advantages.

ETL (Extract, Transform, Load) Architecture

ETL is a traditional data processing pattern where data is extracted from source systems, transformed into a suitable format, and then loaded into a data warehouse or data mart.

- **LinkedIn:** [Deepa Vasanthkumar](#)
- **Medium:** [Deepa Vasanthkumar – Medium](#)

Architectural Patterns in Data Engineering

Key Features: - **Batch Processing:** Typically operates on a schedule (e.g., nightly), processing large volumes of data in batches. - **Data Integration:** Consolidates data from multiple sources into a centralized repository. - **Data Quality:** Ensures data quality through transformation and cleansing steps.

Use Cases: - Building and maintaining enterprise data warehouses. - Consolidating data from disparate systems for reporting and analytics.

Advantages: - **Data Quality:** Provides robust mechanisms for data validation, cleansing, and transformation. - **Centralization:** Creates a single source of truth for enterprise data.

Example Tools: - Apache Nifi, Talend, Informatica, Microsoft SSIS.

ELT (Extract, Load, Transform) Architecture

ELT is a modern variation of ETL where data is first loaded into a data lake or data warehouse, and then transformed as needed.

Key Features: - **Scalability:** Leverages the scalability of modern cloud data warehouses and data lakes. - **Flexibility:** Supports ad-hoc transformations and on-demand processing. - **Real-Time Processing:** Can be adapted for near real-time data processing.

Use Cases: - Cloud-based data warehousing and analytics. - Real-time or near real-time data integration and processing.

Advantages: - **Performance:** Takes advantage of the computational power of modern data warehouses. - **Flexibility:** Allows transformations to be defined and executed dynamically.

Example Tools: - Snowflake, Google BigQuery, Amazon Redshift, Apache Spark.

• **LinkedIn:** [Deepa Vasanthkumar](#) • **Medium:** [Deepa Vasanthkumar – Medium](#)

Architectural Patterns in Data Engineering

Comparing ELT and ETL Architecture

Here is a detailed comparison of ELT (Extract, Load, Transform) and ETL (Extract, Transform, Load) architectures:

Aspect	ETL (Extract, Transform, Load)	ELT (Extract, Load, Transform)
Definition	Data is extracted from source systems, transformed in a staging area, and then loaded into the target data warehouse.	Data is extracted from source systems, loaded into the target data warehouse, and then transformed within the data warehouse.
Process Order	Extract → Transform → Load	Extract → Load → Transform
Transformation	Transformations occur before loading data into the target system.	Transformations occur after loading data into the target system.
Scalability	Limited by the capacity of the ETL server; not as scalable for large datasets.	Highly scalable due to the computational power of modern data warehouses and cloud platforms.
Flexibility	Less flexible; transformations need to be predefined and managed carefully.	More flexible; allows ad-hoc transformations and on-demand processing.
Performance	Performance can be slower due to the need to transform data before loading.	Generally faster for loading data; leverages the performance of the data warehouse for transformations.
Data Volume	Suitable for moderate data volumes; may struggle with very	Ideal for large datasets; can handle vast amounts of data efficiently.

• **LinkedIn:** [Deepa Vasanthkumar](#)

• **Medium:** [Deepa Vasanthkumar – Medium](#)

Architectural Patterns in Data Engineering

Aspect	ETL (Extract, Transform, Load)	ELT (Extract, Load, Transform)
	large datasets.	
Real-Time Processing	Generally better suited for batch processing; real-time capabilities are limited.	Can support real-time or near real-time processing more effectively.
Complexity	Typically more complex to set up and maintain due to the need for a separate transformation layer.	Simpler to set up and maintain; transformation logic is handled within the data warehouse.
Cost	May require additional infrastructure for the transformation stage, potentially increasing costs.	Can be more cost-effective by utilizing the existing capabilities of the data warehouse.
Examples of Use Cases	Building traditional enterprise data warehouses, consolidating data from multiple sources for reporting and analytics.	Cloud-based data warehousing and analytics, real-time data integration, and processing.
Example Tools	Apache Nifi, Talend, Informatica, Microsoft SSIS	Snowflake, Google BigQuery, Amazon Redshift, Apache Spark

Lambda Architecture

Lambda Architecture is designed to handle both batch and stream processing of data. It combines a batch layer for historical data processing with a speed layer for real-time data processing.

Key Features:

- **LinkedIn:** [Deepa Vasanthkumar](#)
- **Medium:** [Deepa Vasanthkumar – Medium](#)

Architectural Patterns in Data Engineering

Batch Layer: Processes large volumes of historical data and generates batch views. -

Speed Layer: Processes real-time data and generates real-time views.

Serving Layer: Merges batch and real-time views for querying.

Use Cases: - Real-time analytics and dashboards. - Systems requiring both historical and real-time data processing.

Advantages: - **Comprehensive Data Processing:** Provides a complete view by combining batch and real-time data. - **Fault Tolerance:** Ensures data availability and consistency through redundancy.

Example Tools: - Apache Hadoop, Apache Spark, Apache Kafka, Apache Storm.

Kappa Architecture

Kappa Architecture is a simplification of Lambda Architecture that processes data streams only, eliminating the batch layer. All data is treated as a real-time stream.

Key Features: -

Stream Processing: All data is ingested and processed as a stream.

Simplicity: Reduces architectural complexity by eliminating the batch layer. -

Reprocessing: Historical data can be reprocessed by replaying streams.

Use Cases: - Real-time data processing and analytics. - Applications where stream processing suffices without the need for a batch layer.

- **LinkedIn:** [Deepa Vasanthkumar](#)
- **Medium:** [Deepa Vasanthkumar – Medium](#)

Architectural Patterns in Data Engineering

Advantages: - **Simplicity:** Easier to implement and maintain compared to Lambda Architecture. - **Real-Time Focus:** Optimized for real-time data processing scenarios.

Example Tools: - Apache Kafka, Apache Flink, Apache Spark Streaming.

Comparing Lambda and Kappa Architecture

Here is a detailed comparison of Lambda and Kappa architectures:

Aspect	Lambda Architecture	Kappa Architecture
Definition	Combines both batch and real-time data processing to provide comprehensive data processing capabilities.	Simplifies data processing by treating all data as a real-time stream. Eliminates the batch layer.
Data Processing Layers	Two layers: Batch layer for historical data processing and Speed layer for real-time data processing.	Single layer: Stream processing for both real-time and historical data.
Batch Layer	Processes large volumes of historical data in batches and generates batch views.	Not present. All processing is done in real-time.
Speed Layer	Processes real-time data and generates real-time views for low-latency access.	Processes both real-time and historical data as streams.
Serving Layer	Merges batch and real-time views to provide a unified view for querying.	Uses stream processing frameworks to query data.

Architectural Patterns in Data Engineering

Aspect	Lambda Architecture	Kappa Architecture
Complexity	More complex due to the need to maintain and synchronize both batch and speed layers.	Simpler, as it eliminates the batch layer and uses a unified stream processing model.
Data Latency	Provides both low-latency real-time views and high-latency batch views.	Focuses on low-latency, real-time data processing.
Scalability	Can handle both large-scale historical data and real-time data streams.	Highly scalable for real-time data processing, may require additional handling for reprocessing historical data.
Reprocessing Historical Data	Can easily reprocess historical data using the batch layer.	Reprocessing historical data involves replaying streams, which can be resource-intensive.
Fault Tolerance	Ensures fault tolerance through redundancy in both batch and speed layers.	Relies on stream processing frameworks for fault tolerance.
Use Cases	Real-time analytics and dashboards requiring both historical and real-time data; complex data processing scenarios.	Real-time data processing applications, such as monitoring, alerting, and streaming analytics.
Performance	Performance can vary depending on the synchronization between batch and speed layers.	Generally offers consistent performance due to its streamlined architecture.
Implementation	Higher, due to the need to manage	Lower, as it focuses on a single,

Architectural Patterns in Data Engineering

Aspect	Lambda Architecture	Kappa Architecture
Complexity	two distinct processing layers and their synchronization.	unified processing model.
Cost	Potentially higher due to the need to maintain separate batch and speed infrastructures.	Can be more cost-effective by reducing the need for separate infrastructures.
Example Tools	Apache Hadoop, Apache Spark, Apache Kafka, Apache Storm	Apache Kafka, Apache Flink, Apache Spark Streaming

Lambda and Kappa architectures serve different needs in data processing. Lambda is suitable for scenarios where both historical and real-time data processing are required, offering comprehensive data views but with added complexity. Kappa simplifies the architecture by treating all data as streams, making it ideal for applications focused on real-time processing with less concern for batch processing. The choice between Lambda and Kappa depends on specific use cases, data latency requirements, and the complexity an organization is willing to manage.

Data Lakehouse Architecture

Data Lakehouse Architecture combines the scalability and cost-efficiency of data lakes with the ACID transactions and data management capabilities of data warehouses.

Key Features: - **Unified Storage:** Stores all types of data (structured, semi-structured, and unstructured) in a single repository. - **ACID Transactions:** Supports ACID transactions for data integrity and consistency. - **Scalability:** Provides scalable storage and compute resources.

Architectural Patterns in Data Engineering

Use Cases: - Unified analytics platforms. - Big data processing and machine learning.

Advantages: - **Versatility:** Supports a wide range of data types and processing workloads.

- **Cost Efficiency:** Reduces the need for separate data storage solutions.

Example Tools: - Delta Lake, Apache Iceberg, Databricks Lakehouse, Snowflake.

Microservices Architecture

Microservices Architecture breaks down data processing into small, independent services that communicate over APIs. Each microservice handles a specific piece of functionality.

Key Features: -

Modularity: Each service is developed, deployed, and scaled independently.

Flexibility: Facilitates the use of different technologies and frameworks for different services.

Resilience: Isolates failures to individual services, improving overall system resilience.

Use Cases: - Complex data processing pipelines. - Scalable and maintainable data processing systems.

Advantages: - **Scalability:** Easily scales individual services based on demand. -

Maintainability: Simplifies development and maintenance through modularity.

Example Tools: - Docker, Kubernetes, Apache Kafka, Spring Boot.

Medallion Architecture in Data Engineering

The Medallion Architecture, also known as the Multi-Hop Architecture, is a data engineering pattern designed to handle large-scale data processing and transformation efficiently. It organizes data into different layers (or “medallions”) to manage and refine the data as it flows through the system. Each layer serves a specific purpose and adds value to the data, ultimately resulting in high-quality, analytics-ready data. The primary goal of Medallion Architecture is to create a structured, scalable, and maintainable data pipeline.

Key Layers in Medallion Architecture

1. Bronze Layer (Raw Data Layer)

- **Purpose:** Ingest raw data from various sources.
- **Characteristics:**
 - Contains raw, unprocessed data.
 - Stores data in its original format.
 - Acts as a single source of truth for ingested data.
- **Examples:** Logs, sensor data, transactional data.

2. Silver Layer (Cleansed Data Layer)

- **Purpose:** Clean, transform, and enrich the raw data.
- **Characteristics:**
 - Contains cleaned and partially transformed data.

Architectural Patterns in Data Engineering

- Handles data validation, deduplication, and normalization.
- Prepares data for more complex transformations and analysis.
- **Examples:** Cleaned logs, normalized transactional data.

3. Gold Layer (Aggregated Data Layer)

- **Purpose:** Aggregate and optimize data for analytics and reporting.
- **Characteristics:**
 - Contains highly processed, aggregated, and optimized data.
 - Data is structured for specific business needs and analytical queries.
 - High data quality, ready for use in BI tools, dashboards, and advanced analytics.
- **Examples:** Aggregated sales data, customer profiles, summary tables.

Advantages of Medallion Architecture

- **Scalability:** Each layer can be scaled independently, allowing for efficient handling of large datasets.
- **Maintainability:** Clear separation of data processing stages makes it easier to maintain and manage the data pipeline.
- **Data Quality:** Progressive refinement of data ensures high data quality in the final analytical layer.
- **Flexibility:** Supports a wide range of data sources and formats, making it adaptable to different use cases.
- **Auditing and Lineage:** Easier to trace data transformations and maintain a history of changes, which is crucial for data governance.

Architectural Patterns in Data Engineering

Example Implementation

Here's an example of implementing Medallion Architecture using PySpark:

```
from pyspark.sql import SparkSession

# Initialize Spark session
spark =
SparkSession.builder.appName("MedallionArchitectureExample").getOrCreate()

# Bronze Layer: Ingest raw data
raw_data_path = "/path/to/raw/data"
bronze_df = spark.read.format("csv").option("header",
"true").load(raw_data_path)
bronze_df.write.format("parquet").mode("overwrite").save("/path/to/bronze_layer")

# Silver Layer: Clean and transform data
bronze_df = spark.read.format("parquet").load("/path/to/bronze_layer")
silver_df = bronze_df.dropDuplicates().filter(bronze_df["value"].isNotNull())
silver_df.write.format("parquet").mode("overwrite").save("/path/to/silver_layer")

# Gold Layer: Aggregate and optimize data
silver_df = spark.read.format("parquet").load("/path/to/silver_layer")
gold_df = silver_df.groupBy("category").agg({"value":
"sum"}).withColumnRenamed("sum(value)", "total_value")
gold_df.write.format("parquet").mode("overwrite").save("/path/to/gold_layer")
```

- **LinkedIn:** [Deepa Vasanthkumar](#)
- **Medium:** [Deepa Vasanthkumar – Medium](#)

Architectural Patterns in Data Engineering

In this example: - **Bronze Layer:** Reads raw data from a CSV file and writes it to a Parquet file. - **Silver Layer:** Cleans the data by removing duplicates and filtering out null values, then writes the cleansed data to another Parquet file. - **Gold Layer:** Aggregates the data by category and writes the aggregated data to a final Parquet file.

The Medallion Architecture provides a structured approach to data processing, ensuring that data is progressively refined and optimized for analytical use. By organizing data into distinct layers, it allows data engineers to manage and maintain large-scale data pipelines effectively, resulting in high-quality, reliable data for business intelligence and analytics. This architecture is particularly well-suited for modern data lakes and cloud-based data platforms, where scalability and flexibility are paramount.

Other Architecture Patterns

Layered (N-Tier) Architecture:

Divides the system into layers with each layer having a specific role, such as presentation, business logic, and data access. Enterprise applications with clear separation of concerns.

Event-Driven Architecture:

Emphasizes the production, detection, consumption, and reaction to events. Applications requiring asynchronous communication and real-time processing.

Architectural Patterns in Data Engineering

Service-Oriented Architecture (SOA):

Focuses on designing software systems that provide services to other applications via a network. Integrating diverse and distributed systems.

Serverless Architecture:

Applications are hosted by third-party services, removing the need for server management by developers. Event-driven applications, microservices, and APIs.

CQRS (Command Query Responsibility Segregation):

Separates read and write operations into different models to optimize performance and scalability. Applications with complex domain models and high read/write loads.

Data Mesh:

A decentralized data architecture emphasizing domain-oriented decentralized data ownership and management. Large organizations with diverse data needs across various departments.

- **LinkedIn:** [Deepa Vasanthkumar](#)
- **Medium:** [Deepa Vasanthkumar – Medium](#)

Architectural Patterns in Data Engineering

Onion Architecture:

Emphasizes a clear separation between the domain model and other aspects of the system, such as user interface and infrastructure. Applications needing strong adherence to domain-driven design principles.

Choosing the right architectural pattern for your data engineering project depends on various factors, including the nature of the data, processing requirements, and organizational goals. As data continues to grow in volume and complexity, leveraging the right architecture will be key to unlocking its full potential for analytics and decision-making.

Data Modeling, Star Schema, Snowflake Schema, Types of Facts, and Dimensions

1. Data Modeling

Definition:

Data modeling is the process of creating a visual representation of data and its relationships to facilitate database design and ensure data integrity, performance, and usability. It helps structure data logically and physically for storage and analysis.

Types of Data Models:

1. Conceptual Data Model:

- High-level, abstract model focused on business requirements.
- Defines entities, relationships, and attributes without technical details.
- Example: Entities like Customer, Order, Product.

2. Logical Data Model:

- Intermediate model that defines the structure of the data, including entity relationships, attributes, and data types, without focusing on the DBMS.
- Example: Customer has attributes like CustomerID (Primary Key), Name, Address.

3. Physical Data Model:

- Implementation-focused, includes DBMS-specific details like table structures, indexes, data types, and constraints.
- Example: MySQL table creation with fields CustomerID INT AUTO_INCREMENT PRIMARY KEY.

2. Star Schema

Definition:

The star schema is a data warehouse schema design that consists of a central **fact table** connected to multiple **dimension tables**. It is optimized for analytical queries and decision-making processes.

Components:

1. Fact Table:

- Central table that contains measurable, numeric data (facts).
- Includes foreign keys referencing dimension tables.
- Example: Sales_Fact table with fields SaleID, DateID, ProductID, CustomerID, and Revenue.

2. Dimension Tables:

- Surround the fact table and store descriptive, textual information about the facts.
- Example: Product_Dimension with fields ProductID, ProductName, Category.

Example Schema:

Fact_Sales	Dimension_Product	Dimension_Customer
SaleID (PK)	ProductID (PK)	CustomerID (PK)
DateID (FK)	ProductName	CustomerName
ProductID (FK)	ProductCategory	Region
CustomerID (FK)		Demographics
Revenue		

Advantages:

- Simplicity: Easy to understand and query.
- Performance: Optimized for read-heavy workloads with fewer joins.

Disadvantages:

- Storage: Denormalized structure leads to redundancy.

3. Snowflake Schema

Definition:

The snowflake schema is a normalized version of the star schema. Dimension tables are split into additional tables to reduce redundancy and storage requirements.

Components:

- Central **fact table** connected to normalized **dimension tables**.
- Example: Instead of storing ProductCategory in the Product_Dimension, create a separate Category_Dimension.

Example Schema:

Fact_Sales	Dimension_Product	Dimension_Category
SaleID (PK)	ProductID (PK)	CategoryID (PK)
DateID (FK)	ProductName	CategoryName
ProductID (FK)	CategoryID (FK)	
CustomerID (FK)		

Fact_Sales	Dimension_Product	Dimension_Category
Revenue		

Advantages:

- Reduces redundancy and storage requirements.
- Better suited for slowly changing dimensions.

Disadvantages:

- Increases query complexity due to more joins.

4. Types of Facts

Facts are numeric measures that represent business metrics.

Categories of Facts:

1. Additive Facts:

- Can be summed across all dimensions.
- Example: Sales_Amount can be totaled by date, region, or product.

2. Semi-Additive Facts:

- Can be summed across some dimensions but not others.
- Example: Account_Balance can be totaled by region but not over time.

3. Non-Additive Facts:

- Cannot be summed across any dimension.
- Example: Ratios or percentages like Profit_Margin.

5. Dimensions

Definition:

Dimensions provide the descriptive context for facts, enabling users to analyze and filter data from various perspectives.

Characteristics:

- Textual and categorical in nature.
- Connected to the fact table via foreign keys.

Examples of Dimensions:

1. Time Dimension:

- Attributes: Date, Week, Month, Year.

2. Product Dimension:

- Attributes: ProductName, Category, Price.

3. Customer Dimension:

- Attributes: CustomerName, Region, Age.

Types of Dimensions:

1. Conformed Dimensions:

- Shared across multiple fact tables or data marts.
- Example: Time_Dimension used in both Sales_Fact and Inventory_Fact.

2. Junk Dimensions:

- Combines unrelated attributes into a single dimension to reduce clutter.
- Example: Flag_Dimension for binary indicators like NewCustomer, PromotionalSale.

3. Degenerate Dimensions:

- Dimension data stored in the fact table itself.
- Example: OrderID in a sales fact table.

4. Role-Playing Dimensions:

- A single dimension table used in different contexts.
- Example: Time_Dimension used as Order_Date and Ship_Date.

5. Slowly Changing Dimensions (SCDs)

Slowly Changing Dimensions (SCDs) are a methodology for handling changes in dimension data over time in a data warehouse while preserving the history of changes where required.

Types of SCDs

1. SCD Type 0 (Fixed Dimensions)

- **Definition:** The dimension data is static and does not change over time.
- **Use Case:** For attributes like Product Launch Date or Social Security Number that must remain constant.

2. SCD Type 1 (Overwrite)

- **Definition:** When a change occurs, the old data is overwritten with the new data, and no history is maintained.
- **Characteristics:**

- Simplest and fastest to implement.
- Suitable for data where historical accuracy is not required.

Example:

CustomerID	CustomerName	Region
101	John Smith	North
101	John Smith	South

3. SCD Type 2 (Versioning)

- **Definition:** Maintains full history by creating a new record for each change in dimension data.
- **Characteristics:**
 - Each record is time-stamped or flagged as active/inactive.
 - Ensures complete historical tracking.

Implementation Options:

1. **Row Versioning:** Add a Version column to identify different versions of the same dimension.
2. **Date Range:** Add StartDate and EndDate columns to define the validity period.

Example:

CustomerID	CustomerName	Region	StartDate	EndDate	CurrentFlag
101	John Smith	North	2023-01-01	2023-06-30	0
101	John Smith	South	2023-07-01	NULL	1

4. SCD Type 3 (Tracking Limited History)

- **Definition:** Maintains limited history by adding columns to store previous values alongside the current value.
- **Characteristics:**
 - Useful when only a small history is needed (e.g., the last two changes).
 - Adds minimal complexity but sacrifices complete historical tracking.

Example:

CustomerID	CustomerName	CurrentRegion	PreviousRegion
101	John Smith	South	North

5. SCD Type 4 (History Table)

- **Definition:** Maintains history in a separate table, while the main dimension table holds only the current data.
- **Characteristics:**
 - Reduces the size of the main dimension table.
 - Separate history table is queried only when historical data is required.

Example:

Dimension Table (Current):

CustomerID	CustomerName	Region
101	John Smith	South

History Table:

CustomerID	CustomerName	Region	StartDate	EndDate
101	John Smith	North	2023-01-01	2023-06-30

6. SCD Type 6 (Hybrid SCD - 1+2+3)

- **Definition:** Combines elements of SCD Types 1, 2, and 3 to track both historical and current data while maintaining versioning.
- **Characteristics:**
 - Adds columns for current and previous values (Type 3).
 - Maintains history in rows (Type 2).
 - Overwrites non-essential fields (Type 1).

Example:

CustomerID	CustomerName	CurrentRegion	PreviousRegion	Version	StartDate	EndDate
101	John Smith	South	North	2	2023-07-01	NULL
101	John Smith	North	NULL	1	2023-01-01	2023-06-30

Choosing the Right SCD Type

SCD Type	When to Use
Type 0	When the data is immutable and never changes.
Type 1	When historical data is irrelevant or unnecessary.
Type 2	When complete historical tracking is critical for analysis and reporting.
Type 3	When only a limited history of changes is required.
Type 4	When maintaining a clean, smaller main table while preserving history is needed.
Type 6	When a combination of history tracking and current data is required.